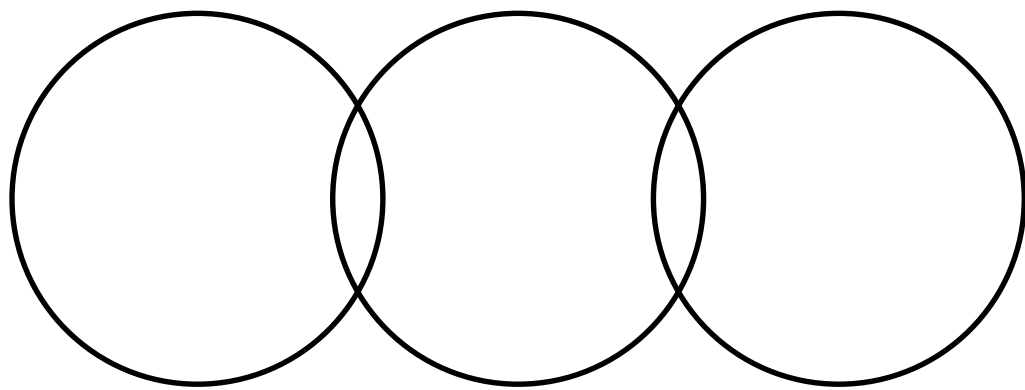
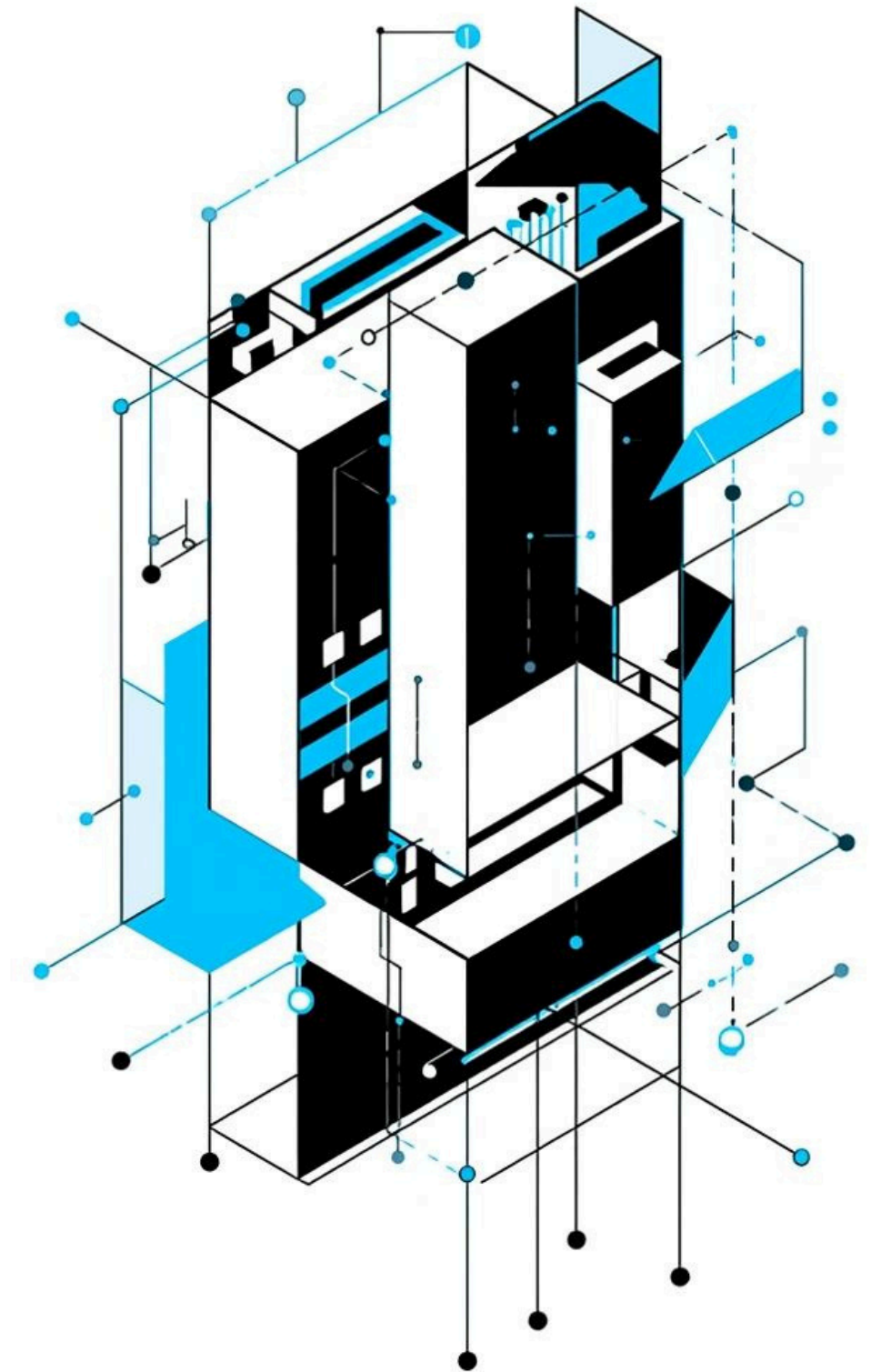




AI AGENT ARCHITECTURE and PATTERN DESIGN



Presented by Tinh To



WHY WE'RE HERE

What is the different?

Standard LLM / Chatbot

The "Brain"

💬 Passive: Waits for a prompt.

📖 Output: Generates Text/
Images.

🧠 Capability: "I know how to
write SQL."

AI Agent

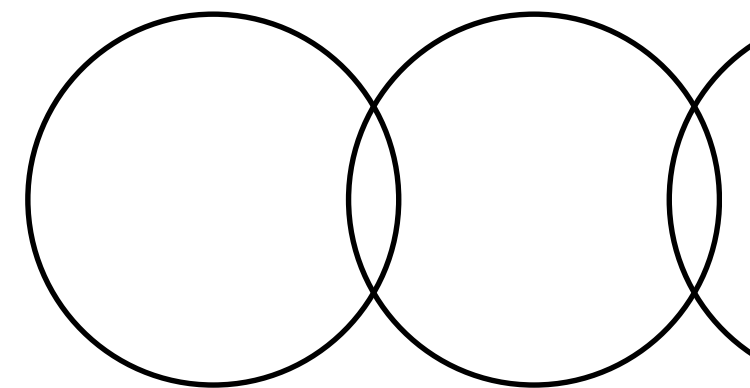
The "Brain" + "Hands" + "Mind"

⚡ Active: Works towards a goal.

🔧 Output: Executes Actions (APIs,
DBs).

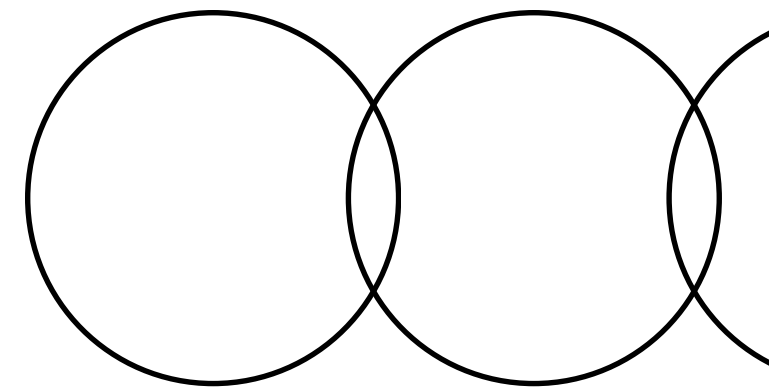
🚀 Capability: "I connect to the DB
and run the query."

Chatbot talks, Agent does.



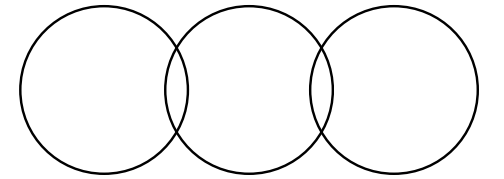
THE AGENDA

- | See the shape - Look at work and know what kind of solution fits
- | Think in components - Understand what an agent needs
- | Choose the pattern - Chain, DAG, Agentic or Hybrid
- | Design for humans - Know where people stay in the loop



THE MINDSET SHIFT

Let's Clear Some Things Up



MISCONCEPTION

- | AI replaces worker
- | Just add AI to everything
- | Agents are magic
- | More autonomous = better
- | One architecture fits all
- | Humans slow things down

REALITY

- | AI amplifies workers
- | Start with the work, not the tech
- | Agents are system with parts
- | Match autonomy to the work
- | Different shapes, different patterns
- | Human are quality gates

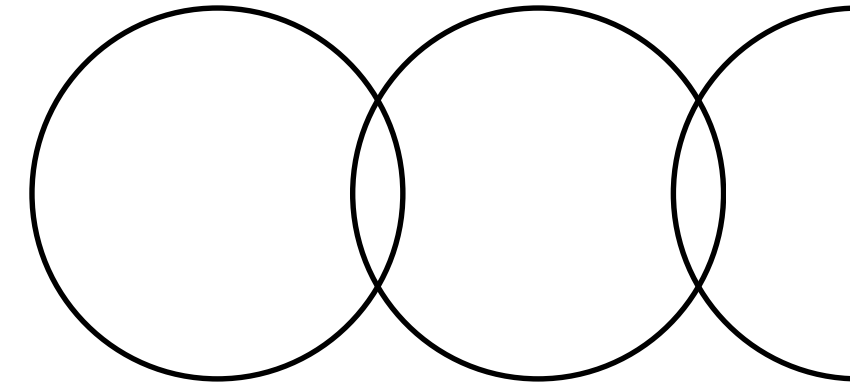
“

"Agents **amplify** work,
they don't replace workers."

Same people. More leverage.

THE DISASTERS

"70% of AI agent projects fail to reach production (Gartner, 2024)"



SITUATION

Requirements
Code
Business

- *35% fail in pilot phase*

- *15% killed by compliance/security concerns*

PROBLEM

Misunderstand
Bug
Mistakes

- *25% can't scale beyond proof-of-concept*

- *5% "succeed" but nobody uses them*

RESULT

Failure
Crash
Learn

- *25% can't scale beyond proof-of-concept*

DISASTER 1

Understanding the promise and its pitfalls

PROBLEM

Unfortunately, the **promised feature did not exist**, leading to significant **miscommunication** and **unmet client expectations**.

SITUATION

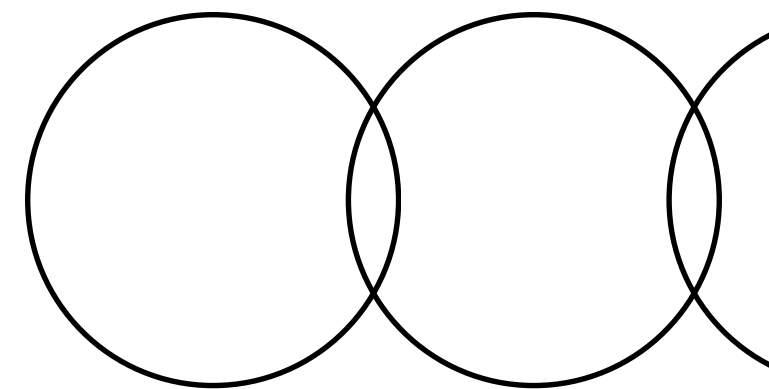
The proposal agent was tasked with drafting a client communication, **promising seamless real-time synchronization capabilities**.

RESULT

This failure resulted in a **lawsuit against the company**, ultimately leading to the **loss of a valuable client account**.

THE AGENT DIDN'T KNOW

IT WAS CONFIDENTLY WRONG



THE BRAIN



THE MODEL

WHERE

Cloud — *Frontier intelligence, data leaves premises*

Local — *Full privacy, less capable models*

HOW

Fast models — *Follow instructions, quick, cheap*

Thinking models — *Reason through problems, slower*

SIZE

Small models - *Faster, cheaper tailor for simple repeat tasks*

Large models - *Smart, expensive big knowledge*

CONTEXT WINDOW

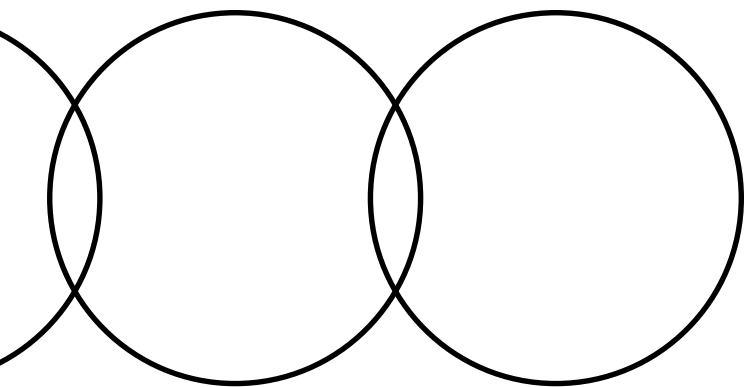
More context window more data to digest

DISASTER 2

The Consequences of Misidentifying Users

PROBLEM

The agent found a user named Sarah, proceeded to reset the password, and ended up locking the wrong person out.



SITUATION

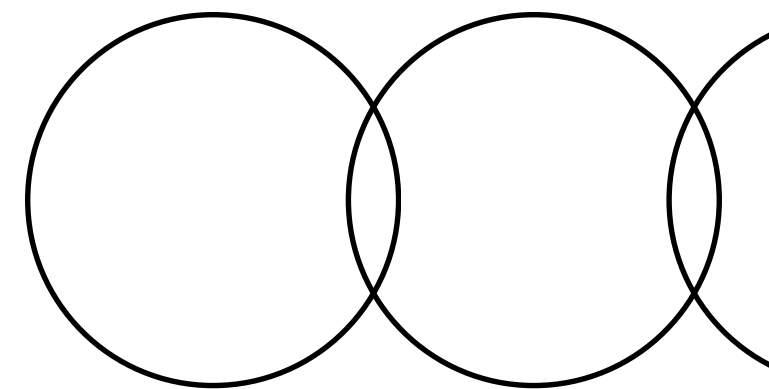
An IT helpdesk ticket requested a password reset for a user named Sarah without confirming the specific individual involved.

RESULT

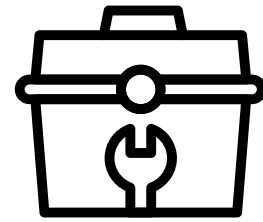
The **CEO's Executive Assistant was locked out**, jeopardizing their access just before a critical board meeting in ten minutes.

WHICH SARAH WAS IT?

THE AGENT COULDN'T ASK



THE HANDS



The tools

What can it DO?

Query database? Read tickets? Parse PDF? Ask question?

What needs **CONFIRMATION**?

If an action yields impacts, always ask for confirmation.

What's the **BLAST RADIUS**?

READ is safe

CREATE sometimes is not

UPDATE and DELETE is definitely not

What breaks if wrong?

Identify where the crash point is

DISASTER 3

Understanding the Blank Check Scenario in AI

PROBLEM

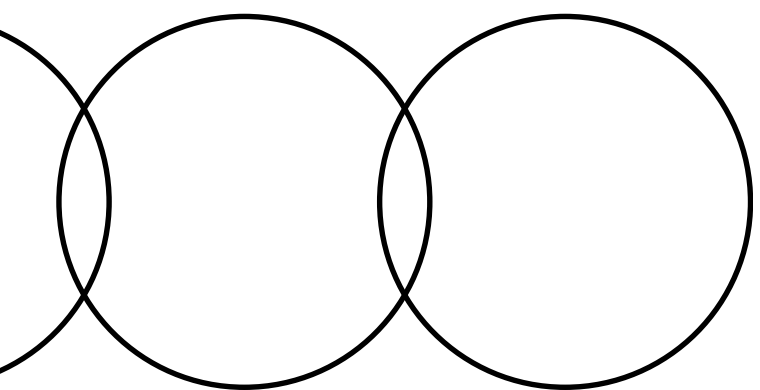
A team member submitted a **\$47,000 'team dinner'** expense. The agent **approved it without scrutiny**, leading to inevitable complications.

SITUATION

The expense agent processes **200 reports per week**, automating transactions to streamline operations. A high volume introduces significant risks.

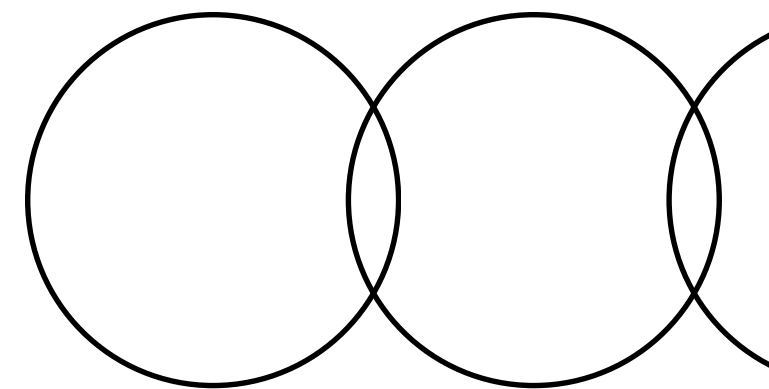
RESULT

The agent validated all parameters: receipt, category, format — **resulting in automatic approval**, highlighting the dangers of unchecked automated systems.



THE AGENT'S LIMITATIONS

Understanding its boundaries



WHAT IT SHOULDN'T SEE

...

WHAT IT SHOULDN'T DO

...

WHAT IT SHOULDN'T KNOW

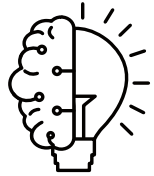
...

AGENTS SHOULD ALWAYS ON MONITORING

Also, a comprehensive checkpoint and backup is a must.

THE FIVE COMPONENTS

BRAIN



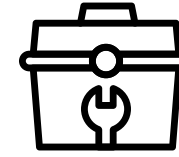
The brain is where an agent's **thinking** occurs, enabling it to process information and make informed decisions.

SENSES



Senses allow agents to **perceive** their environment by gathering data through various inputs, such as sensors and information feeds.

HANDS



The hands represent the agent's ability to **act** on decisions, executing tasks that were analyzed and planned.

GUARDRAILS



Guardrails ensure agents operate within **constraints**, providing boundaries that prevent harmful or undesirable actions from occurring.

MIND



The mind encompasses the agent's **knowledge**, integrating past experiences and learned information to improve future decision-making processes.

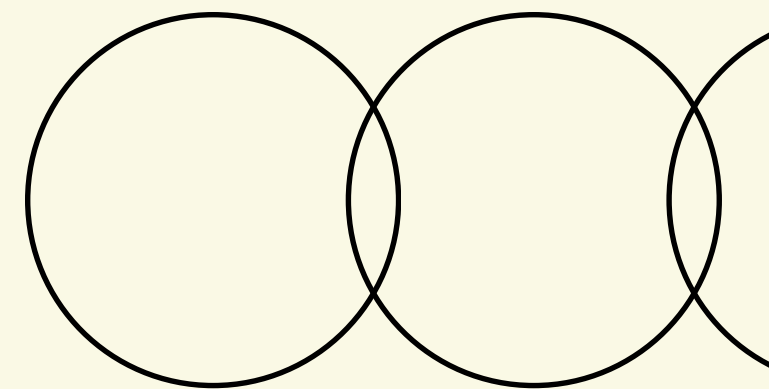
“

"Imagining LLMs like a **very**
large database with **poor**
design."

*"We wouldn't want to touch it if it weren't so bloody
useful."*

BREAK TIME

Take a short pause



THE PATTERN MISTAKE

WRONG PATTERN = DISASTER

MegaCorp saw a **competitor** automate support with an autonomous agent.

Fully agentic: Think, act, observe, adapt.

MegaCorp built the same thing. For their support process.

Their process: verify identity → check account → apply standard fix → close ticket.

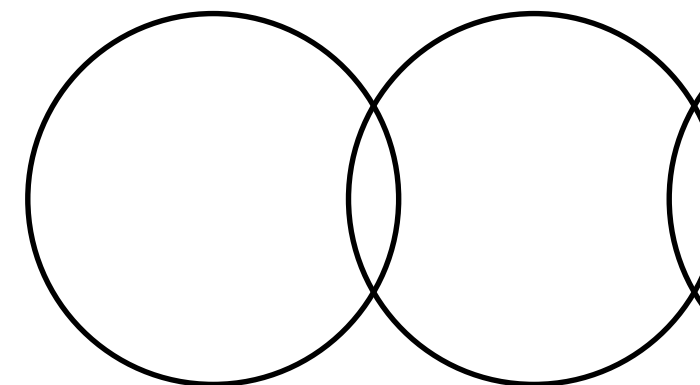
Same steps. Every time. 200 tickets a day.

The autonomous agent "*thought*" about each ticket. Sometimes found **creative solutions nobody asked for**. Sometimes went in circles. Sometimes gave refunds it shouldn't have.

Handle time went up. Answers got **inconsistent**.

They built a thinking machine for work that didn't need thinking.

Wrong pattern for the work = disaster even with perfect components.



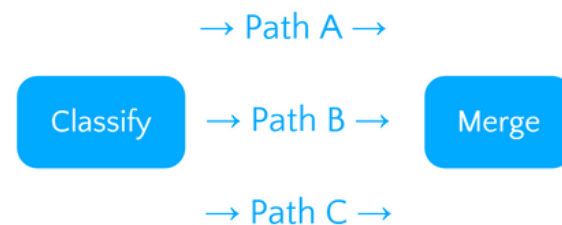
HOW TO CHOOSE

CHAIN



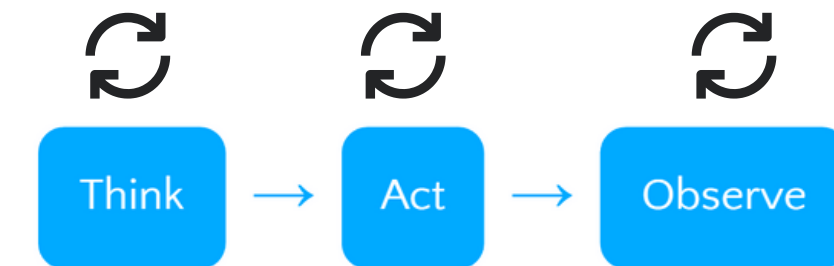
This method ensures a **fixed sequence**, where each step follows the previous one. It's highly predictable and efficient for routine tasks.

DAG



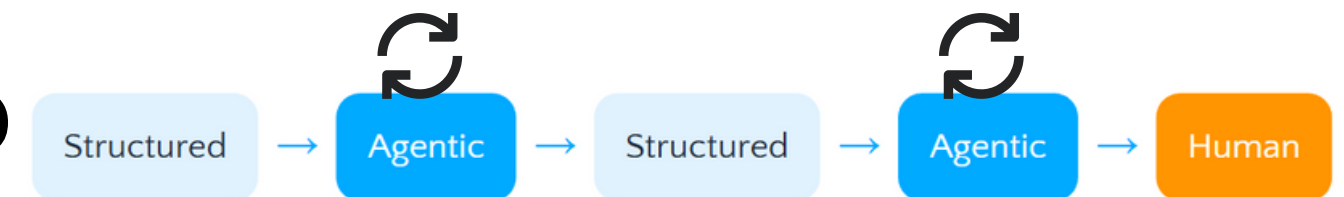
Directed Acyclic Graphs (DAG) allow for paths that **branch based on input**. They handle complex workflows with varying steps dynamically and intelligently.

AGENTIC



In this approach, agents can **think and adapt**. They continuously observe and learn from their environment, allowing for flexible decision-making.

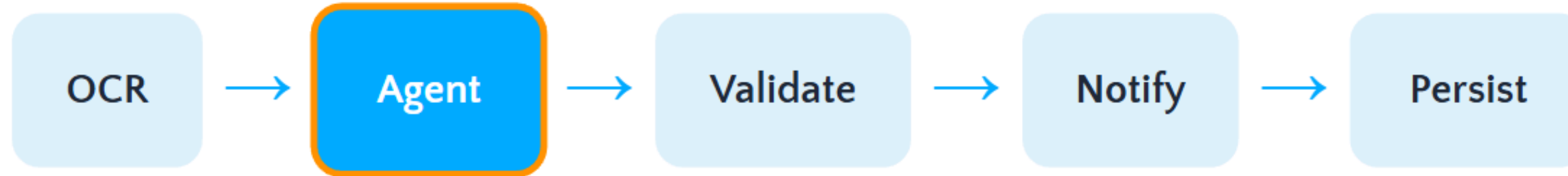
HYBRID



A Hybrid model combines elements of **fixed paths and adaptive decision-making**, making it ideal for tasks that require both structure and creativity.

CHAIN SEQUENCE IN AI

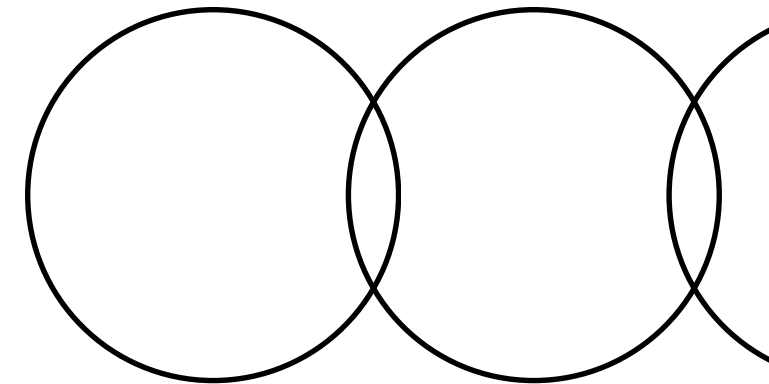
Fixed Steps Every Time



```
from pydantic_ai import Agent, RunContext
# Knowledge: expense policies from vector DB
expense_agent = Agent(
    'openai:gpt-4o',
    deps_type=ExpenseKnowledge, # RAG context
    output_type=ExpenseData,
)

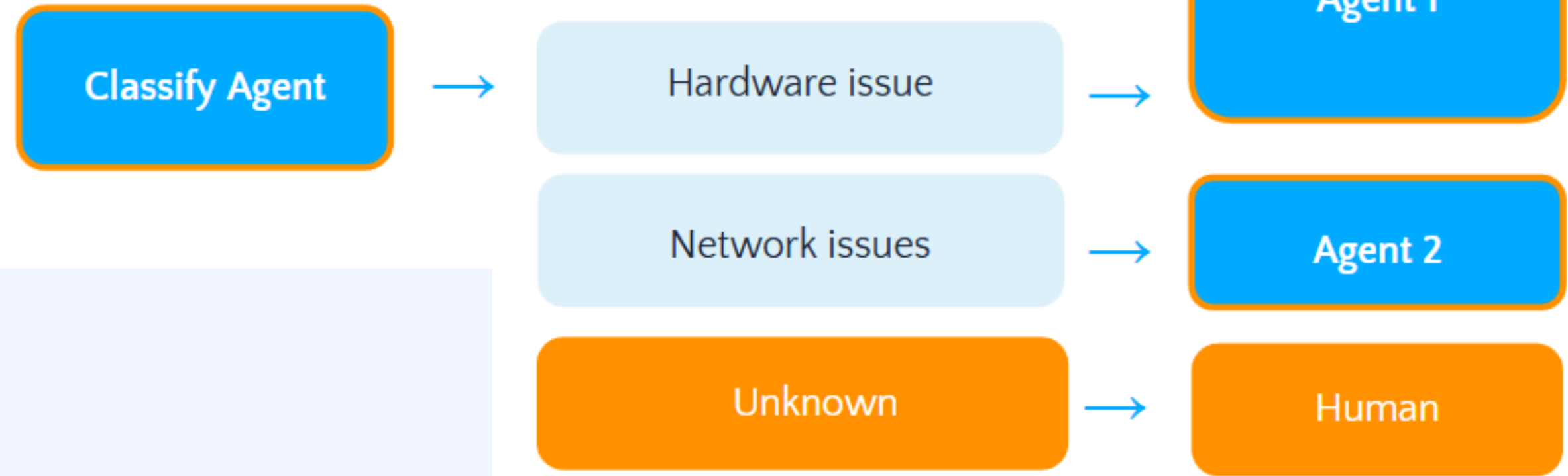
@expense_agent.system_prompt
def add_policies(ctx: RunContext[ExpenseKnowledge]) -> str:
    return f"Policies: {ctx.deps.get_policies()}"

# Workflow
def process_expense(doc):
    knowledge = ExpenseKnowledge(vector_db)
    result = expense_agent.run_sync(ocr(doc), deps=knowledge)
    if result.output.amount > 5000: human_review()
    persist(result.output)
```



THE DAG METHODS

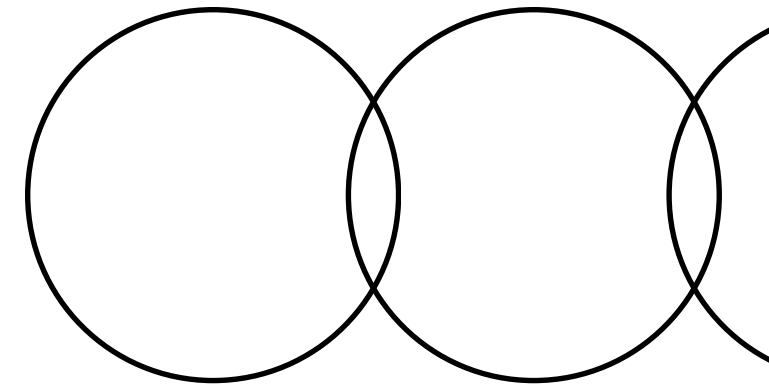
ROUTE PROBLEM TO CORRECT SOLVERS



```
from pydantic_graph import BaseNode, Graph
# Knowledge: troubleshooting guides, system docs
classifier = Agent(
    'deepseek:deepseek-chat',
    deps_type=ITKnowledgeBase, # RAG: guides, docs
    output_type=TicketCategory,
)

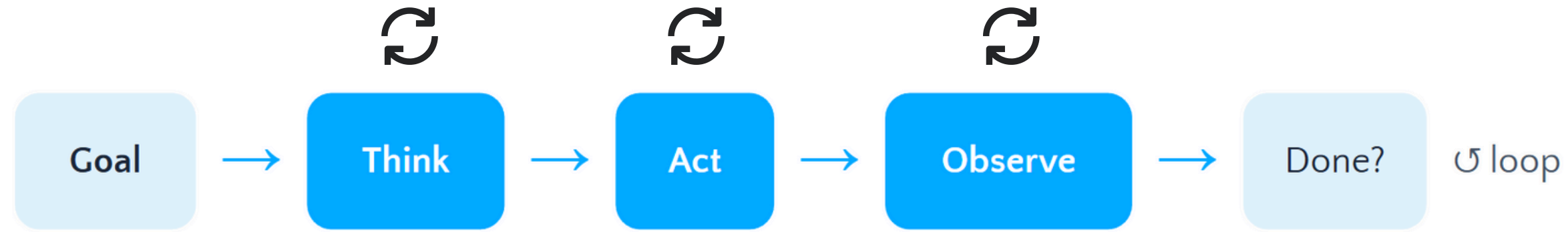
@classifier.system_prompt
def add_guides(ctx: RunContext[ITKnowledgeBase]) -> str:
    return f"Guides: {ctx.deps.search_guides()}"

@dataclass
class ClassifyNode(BaseNode):
    async def run(self, ctx) -> PasswordNode | HardwareNode | UnknownNode:
        kb = ITKnowledgeBase(vector_db)
        result = classifier.run_sync(self.ticket, deps=kb)
```



THE AGENTIC SYSTEM

THINK, ACT, OBSERVE, REPEAT



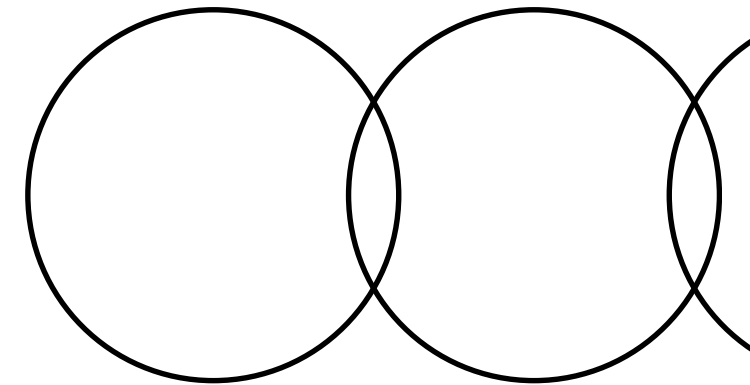
```
# Knowledge: internal reports, market data, news
research_agent = Agent(
    'anthropic:claude-sonnet-4-0', # best for reasoning
    deps_type=ResearchKnowledge, # RAG context
)

@research_agent.system_prompt
def add_context(ctx: RunContext[ResearchKnowledge]) -> str:
    return f"Internal data: {ctx.deps.get_reports()}"

@research_agent.tool # Agent has tools -> agentic
def search_internal(ctx: RunContext, query: str) -> list:
    return ctx.deps.vector_search(query)

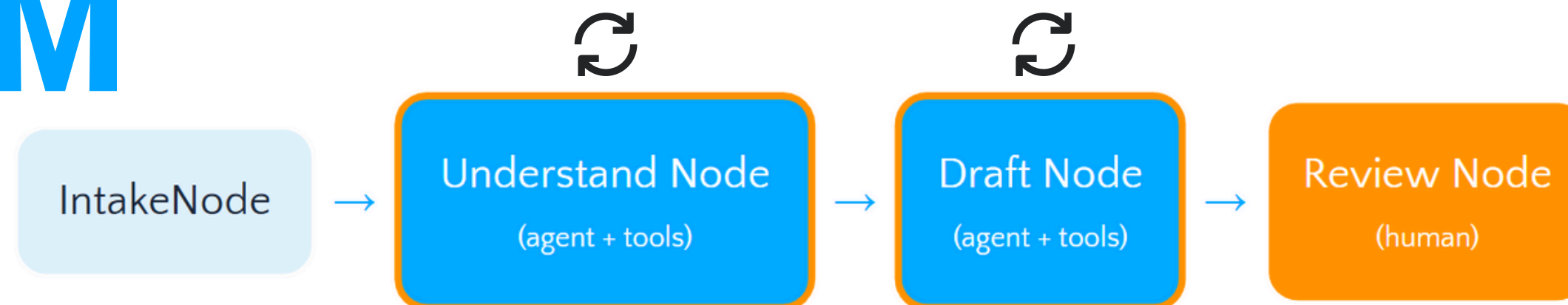
@research_agent.tool
def search_market(query: str) -> list: ...

# Agent loops: search -> analyze -> search more -> synthesize
result = research_agent.run_sync(goal, deps=ResearchKnowledge(db))
```



HYBRID SYSTEM

ALL OF THEM



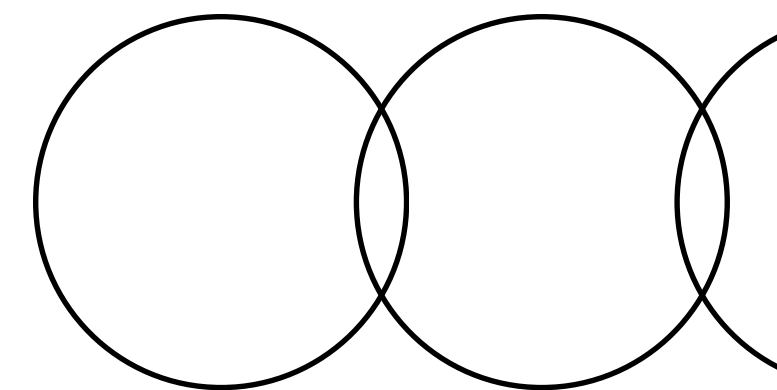
Simple node Agentic node (agent + tools) Human node

```
# Knowledge: case studies, pricing, product specs
proposal_agent = Agent(
    'openrouter:gwen/gwen3-235b-a22b',
    deps_type=SalesKnowledge, # RAG: cases, pricing
)

@proposal_agent.system_prompt
def add_context(ctx: RunContext[SalesKnowledge]) -> str:
    return f"Products: {ctx.deps.get_specs()}"

@proposal_agent.tool # Has tools -> agentic in graph
def find_cases(ctx: RunContext, industry: str) -> list:
    return ctx.deps.search_cases(industry)

@dataclass
class DraftNode(BaseNode): # Graph node with agentic agent
    async def run(self, ctx) -> ReviewNode:
        result = proposal_agent.run_sync(self.brief, deps=SalesKnowledge(db))
```



“

“THERE IS NO **ONE-SIZE-FITS-
ALL SOLUTION**”

*“Also, no perfect solution, just good enough
ones.”*

THE REMOVAL DISASTER

SpeedCorp wanted maximum efficiency. Customer refund agent. Fully automated.

| Request → Check policy → Decide → Process → Done.

Fast. Scaled beautifully.

For three weeks,

Then:

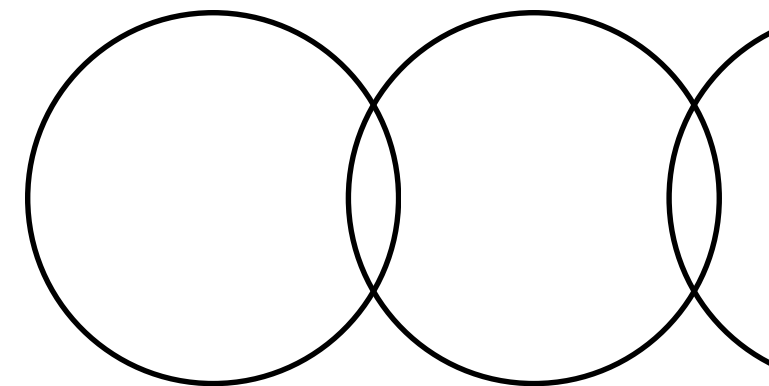
Subscription canceled by mistake — agent refunded and closed account. Fraudster found loophole - 47 fake refunds.

Grieving widow needed account transfer — agent said "policy doesn't allow" and closed ticket.

No human ever saw these.

Agent followed rules.

Rules didn't cover everything.



WHAT'S THE CATCH?

AGENTS HANDLE VOLUME

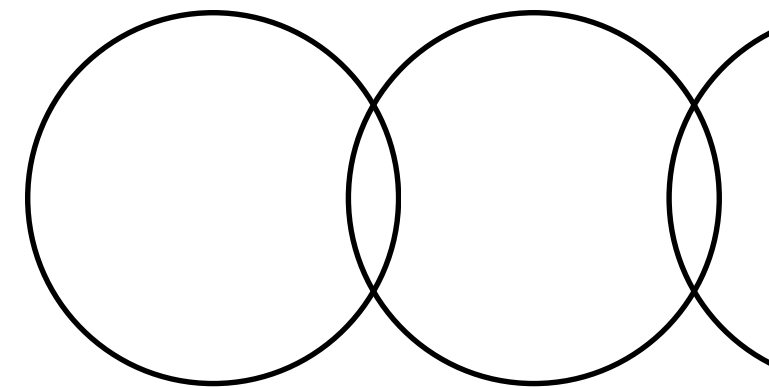
In the realm of AI, agents excel at **managing high volumes** of tasks efficiently, yet it's the human touch that **ensures sound judgment** and critical thinking in complex situations.

TRUST ISSUES

AI shouldn't ever be fully trusted.

THE RESPONSIBILITY

AI makes a mistake, humans take responsibility.



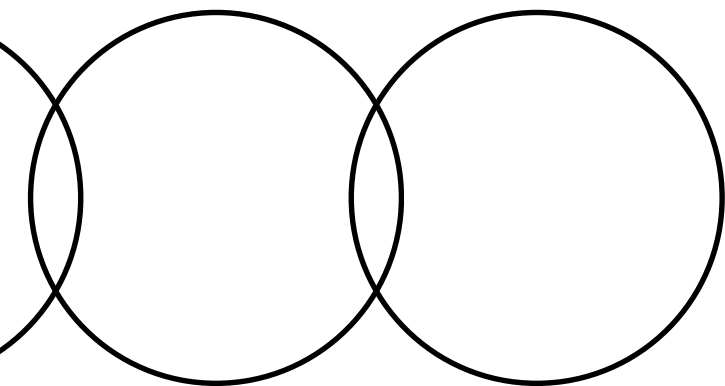
THREE ROLES

Defining how humans interact with AI agents

COLLABORATOR

Collaborators engage with AI agents during work, enhancing creativity and providing **guidance** for tasks that require nuanced understanding.

*Human **in** the loop*



SUPERVISOR

Supervisors play a crucial role by approving actions **before decisions** are made, ensuring high-stakes choices have human oversight.

*Human **on** the loop*

MONITOR

Monitors evaluate the outcomes of AI actions **after execution**, auditing results to ensure compliance and accuracy in routine processes.

Observability

“

"AGENTS handle VOLUME.
HUMANS handle JUDGMENT."

*Together, they create a balanced and effective
system.*

BEFORE YOU BUILD

Make a blueprint

WHAT BUSINESS AGENT LEVERAGE?

....

HOW USER WORK?

...

WHERE IS THE CRASH POINT

...

AGENT COMPONENTS

BRAIN:

SENSES:

HAND:

MIND:

GUARDRAILS:

WHAT SHAPE?

CHAIN:

DAG:

AGENTIC:

HYBRID:

WHERE HUMANS?

HUMAN ON THE LOOP:

HUMAN IN THE LOOP:

OBSERVABILITY:

Let's play a game



LET BUILD ONE,

LIKE RIGHT NOW

What **business challenge** are you looking to **?**
address with agents

LET BUILD ONE AGENT ROUND 1

Brain "What kind of thinking does this need?"

- Fast model (rule-following, extraction) or thinking model (reasoning, judgment)?
- Where might it be confidently wrong?

Mind "What does it need to know?"

- What knowledge does it need? (Policies? History? Docs?)
- What does "enough" look like?
- When should it stop searching?

Senses "What does it see? What does it say?"

- What's the input? (Form? Email? Image? Voice?)
- What's the output? (Decision? Draft? Action?)
- What should it never see or share?

Guardrails "What should it never do alone?"

- Hard limits?
- When must a human decide?
- What's the kill switch?

Hands "What tools does it need?"

- What systems does it touch?
- What actions can it take?
- What's the blast radius if a tool fires wrong?

LET BUILD ONE AGENT ROUND 1

Brain "What kind of thinking does this need?"

<MODEL>

Senses "What does it see? What does it say?"

<INPUT>

Hands "What tools does it need?"

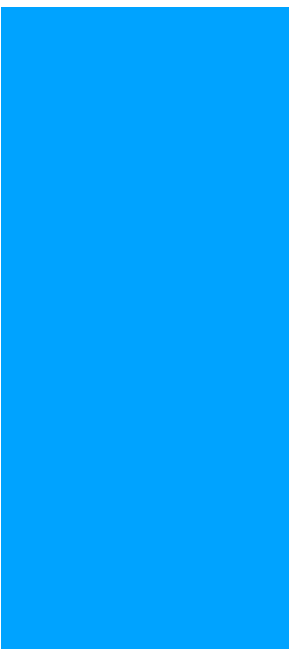
<TOOLS>

Mind "What does it need to know?"

<KNOWLEDGE>

Guardrails "What should it never do alone?"

<CONSTRAINT>

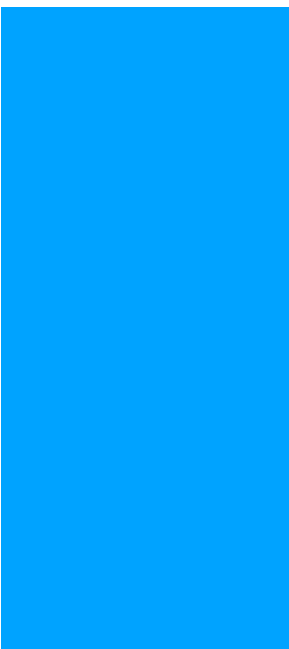


LET BUILD ONE AGENT ROUND 2

Pattern "What's the shape?"

- Chain? Same steps every time → What are the steps? In what order?
- DAG? Branches based on input → What are the branches? What triggers each?
- Agentic? Exploration, unknown path → What's the think-act-observe loop? When does it stop?
- Hybrid? Mix → Which parts are fixed? Which parts think?

Humans "Where do people stay in the loop?"

- Supervisor (before action)?
 - Collaborator (during)?
 - Monitor (after)?
 - Where exactly?
- 

LET BUILD ONE AGENT

ROUND 2

Pattern "What's the shape?"

...

Humans "Where do people stay in the loop?"

...



LET BUILD ONE AGENT REVIEW

THE ARCHITECT:

MODEL:

INPUT:

OUTPUT:

TOOLS

KNOWLEDGE:

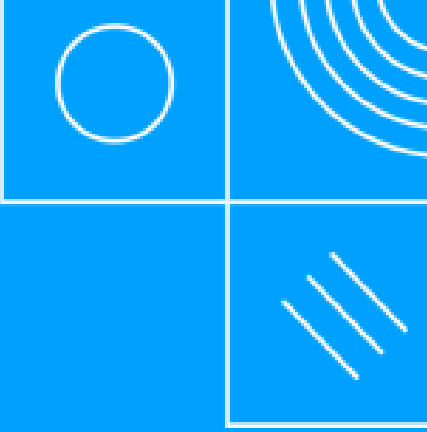
GUARDRAILS

THE DESIGN:

PATTERN:

HUMAN ASPECT:





Q&A

THANK YOU!

